

Recap

HTTP header fields can be used by web developers to define privacy and security policies aimed at restricting the actions permitted by browsers and web servers

These policies improve the resilience of web applications against common security threats (e.g., XSS)

Some policies are implemented by browsers (e.g., SOP), while some others (e.g., CORS) by web servers

Web performance depends on many factors related to web design, network and web server characteristics and load and traffic conditions

Caching mechanisms are aimed at making the web faster by storing HTTP response messages “closer” to the users, thus reducing the network traffic and the web server load

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

69

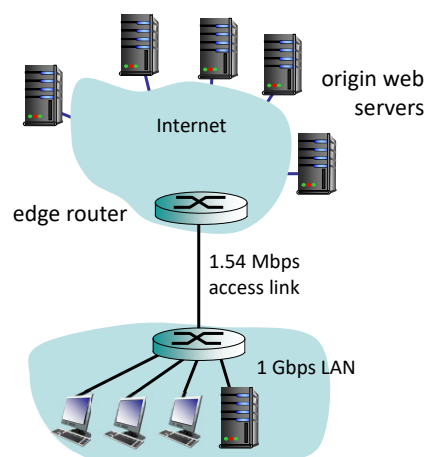
Example

Baseline scenario:

- Request rate from clients to origin servers: 15/sec
- Object size per request: 10^5 bits
- LAN bandwidth: 1 Gbps
- Access link bandwidth: 1.54 Mbps
- Delay between edge router and any origin web server: 200 msec

Assumption: HTTP requests and responses are small and do not create significant traffic on the network

Estimation of the time to download an object



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

70

Baseline scenario: utilization

Network utilization:

- fraction of bandwidth usage
- number between 0 and 1

$$Utilization = \frac{Data_rate}{Bandwidth} = \frac{Request_rate * Object_size}{Bandwidth}$$

$$LAN\ Utilization = \frac{15 * 10^5}{1 * 10^9} = 0.0015$$

$$Link\ Utilization = \frac{15 * 10^5}{1.54 * 10^6} = 0.974 \quad \text{Bottleneck}$$

$Utilization \approx 0 \rightarrow$ small delays

$Utilization \approx 1 \rightarrow$ very large delays

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

71

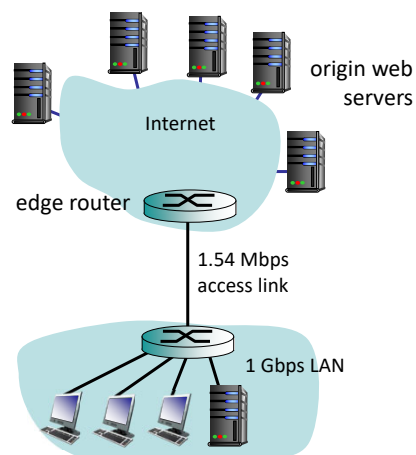
Baseline scenario: performance results

Qualitative performance:

- LAN utilization: 0.0015
- Access link utilization = 0.974

Total time =

Internet delay + access link
delay + LAN delay =
200 msec + minutes + μ sec



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

72

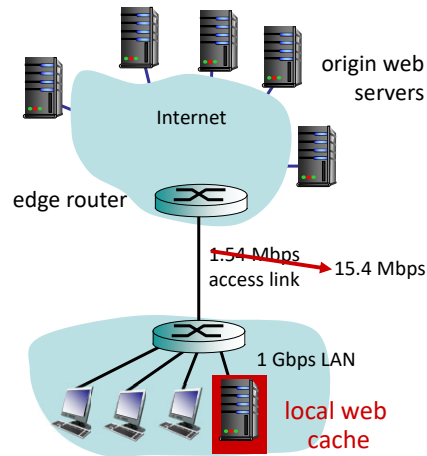
Possible solutions

Scenario 1:

- Upgrade the access link by increasing the bandwidth from 1.54Mbps to 15.4Mbps
- Bottleneck is removed
- Performance improves but it is a costly solution

Scenario 2:

- Install a local web cache
- Cheaper solution
- Link utilization depends on hit/miss ratio
- Performance depends on hit/miss ratio



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

73

Caching

Not every web object can be cached

The properties of an object with respect to caching are defined by webmasters who insert appropriate directives in the web server configuration files

These directives are “translated” into the corresponding header fields included in HTTP response messages

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

74

Cache-control header field

The **Cache-control** header field specifies caching directives

- **public**: HTTP response message is marked as cacheable
- **private**: HTTP response message intended for a single user (private caches only)
- **no-cache**: HTTP response message can be stored but cannot be reused (revalidation)
- **no-store**: HTTP response message **must not** be stored in any cache (e.g., copyrighted material, dynamic content)

These directives are necessary but not sufficient to ensure data confidentiality (e.g., compromised/malicious caches can ignore the directives and store response messages)

Other directives can be associated with the **Cache-control** header field to specify additional caching properties

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

75

Cache directives in HTTP request messages

Directives about caching can also be included in HTTP request messages by clients to request a specific behavior about caching

These directives are client “advisories” specified in browser settings

Web servers are not required to comply with them

The **Cache-control** header field specifies these directives

- **no-cache**: the client prefers a stored response not be used to satisfy the request without successful validation on the origin server
- **no-store**: caches must not intentionally store information in permanent storage and must make a best effort attempt to remove the information from RAM as fast as possible after forwarding it

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

76

HTTP response validity

The HTTP response messages stored in the cache can be

- fresh, i.e., valid (can be reused)
- stale, i.e., invalid (cannot be reused)

Before providing a response to a client, the freshness of the response message needs to be assessed (i.e., cache hit: HTTP response available in cache **and** fresh)

This assessment is based on the information provided by web servers in the HTTP response message

Two possible solutions:

1. Expiration mechanisms
2. Validation mechanisms

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

77

Expiration mechanisms

An HTTP response message can contain explicit expiration directives specifying the lifetime in the cache, thus making it very easy for a cache to assess whether the response is fresh or stale

Cache-control:, **max-age=...**

Expires: date & time

```
HTTP/2 200
date: Thu, 30 Apr 2026 08:24:11 GMT
content-type: text/html; charset=UTF-8
server: cloudflare
cache-control: public, max-age=14400
x-drupal-dynamic-cache: UNCACHEABLE (poor cacheability)
content-language: en
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
vary: Cookie,Accept-Encoding
expires: Thu, 30 Apr 2026 12:24:11 GMT
x-drupal-cache: HIT
last-modified: Thu, 30 Apr 2026 07:59:21 GMT
cf-cache-status: MISS
x-varnish: 211947941
grace: none
x-cache: MISS
server-timing: cfCacheStatus;desc="MISS"
server-timing: cfEdge;dur=46,cfOrigin;dur=149
cf-ray: 9f4526105fbb9b-MXP
```

max-age
preferred

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

78

Expiration mechanisms (cont'd)

Expiration times in the past or **max-age=0** indicate that the response is to be considered stale (i.e., the web server forces a new HTTP request)

```
HTTP/1.1 301 Moved Permanently
Date: Thu, 30 Apr 2026 08:30:21 GMT
Server: Apache/2.2.16 (Debian)
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
Link: <http://webing.unipv.eu/wp-json/>; rel="https://api.w.org/"
Location: http://webing.unipv.eu
Vary: Accept-Encoding
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
X-Content-Encoding-Over-Network: gzip
Transfer-Encoding: chunked
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

79

No expiration details

Not all HTTP response messages specify information about their expiration

Heuristic expiration time is assigned by a cache when no explicit expiration time is available

Heuristics estimate a “reasonable” expiration time using values of other header fields, e.g., **Last-Modified** date and time

A heuristic expiration value could be set to a fraction of the time since object was last modified (e.g., 10%)

```
HTTP/1.1 200 OK
Date: Thu, 30 Apr 2026 08:41:21 GMT
Server: Apache/2.4.58
Last-Modified: Thu, 06 Jun 2024 20:17:57 GMT
ETag: "f4e-61a3e60fec95d-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=99
Connection: Keep-Alive
Content-Type: text/css
X-Content-Encoding-Over-Network: gzip
Transfer-Encoding: chunked
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

80

Header settings

Specific web server components used for header field definitions (e.g., Apache `mod_headers`, `mod_expires`; Nginx `ngx_http_headers_module`)

Apache

```
Header set Cache-Control "no-cache, must-revalidate"
```

```
ExpiresActive On
```

```
ExpiresByType text/* "access plus 2 months"
```

```
ExpiresByType application/* "access plus 2 minutes"
```

```
ExpiresByType image/* A0
```

Nginx

```
location ~* \.(css|js)$
```

```
{ expires 1m;
```

```
add_header Cache-Control "public";
```

```
}
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

81

Expiration advantages & disadvantages

Expiration mechanisms are useful for content that seldom changes (e.g., logos, style sheets)

- + Significant improvement of the PLT
- Poor flexibility for webmasters (content cannot be modified to avoid inconsistencies)

No visits to the websites mean no (customized) advertisements and no revenues for service providers

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

82

Validation mechanisms

The origin web server has the master copy of the web object

The idea is to transform a potentially stale response stored in the (browser or proxy) cache into a fresh response by asking the origin server

This mechanism is known as validation (or revalidation)

Stale responses are not immediately discarded by the browser

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

83

HTTP response messages

Each HTTP response message includes specific information about the object being transferred:

- Date and time of the object last change (**Last-Modified:**)
- An opaque arbitrary string uniquely identifying the object version i.e., entity tag (**Etag:**)
- **Last-Modified:** is also used for other purposes (e.g., to adjust crawling frequency)

```
HTTP/1.1 200 OK
Date: Thu, 30 Apr 2026 08:41:21 GMT
Server: Apache/2.4.58
Last-Modified: Thu, 06 Jun 2024 20:17:57 GMT
Etag: "f4e-61a3e60fec95d-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=99
Connection: Keep-Alive
Content-Type: text/css
X-Content-Encoding-Over-Network: gzip
Transfer-Encoding: chunked
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

84

Conditional HTTP requests

A conditional HTTP request is used to check whether the object stored by the server matches the version stored in the cache

A conditional request is an HTTP request with one or more request header fields that indicate a pre-condition to be tested before applying the request method to the target object [[RFC 9110 “HTTP Semantics”](#) published in June 2022]

The request message includes a validator (e.g., last modification of the object, entity tag) that indicates the version of the cached object

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

85

Conditional header fields

If-Modified-Since: ←
If-None-Match: ←
. . . .
Last-Modified: Thu, 06 Jun 2024 20:17:57 GMT
ETag: "f4e-61a3e60fec95d-gzip"

These headers define a pre-condition

The outcome of the request depends on the pre-condition:

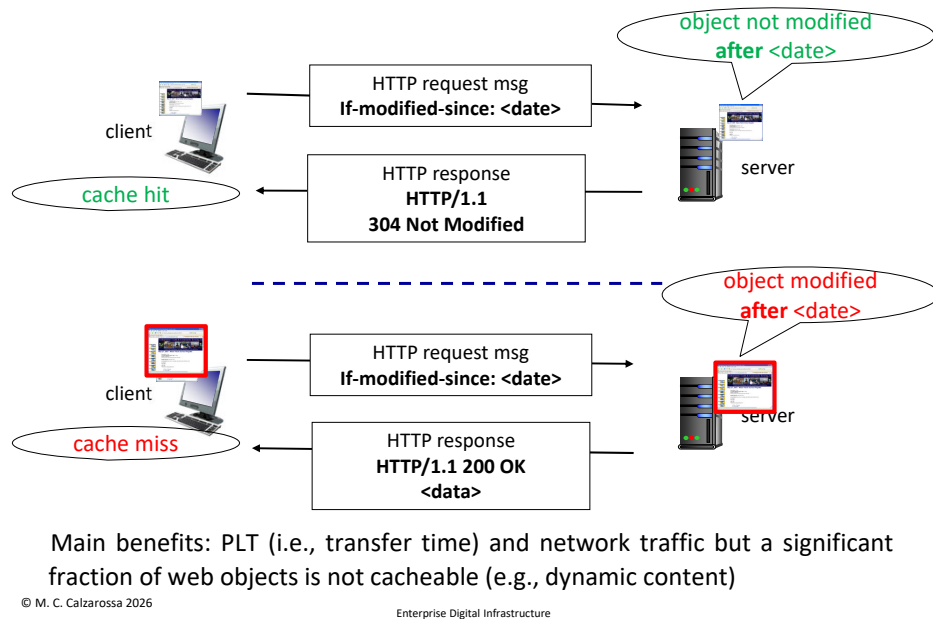
- Precondition matched: object is fresh
 - No need to transfer the object (already in cache)
 - HTTP response message with status code **304 Not Modified** [empty body section]
- Precondition not matched: object is stale
 - Object must be transferred in the HTTP response message
 - HTTP response message with status code **200 OK**

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

86

Conditional requests: implementation



87

Strong vs weak validation

A webmaster can specify the type of validation to be performed using the **Etag**: header

Two types of validation:

- Strong validation, i.e., two versions of an object are identical if their content is byte to byte identical

`etag: "fdbded63b983c2e62692186adc4638cd:1681916649.287007"`

- Weak validation, i.e., two versions of an object are identical if their content is equivalent

`etag: W/"eb5bfe30bacf78fb9411466f1200979b"`

Weak validation is complex to implement but is very useful for optimizing cache performance

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

88

Summary of expiration & validation

Expiration is an explicit mechanism implemented by webmasters; browsers assess the validity of cached responses without interacting with web servers

Validation is an implicit mechanism that requires interactions of clients with web servers

Client side: HTTP response message corresponding to the specified URI is stored in the browser cache but it does not include any expiration detail

- Client starts the validation process by sending an HTTP request message that includes “**IF-**” header fields

Server side: server receives the HTTP request message and checks condition(s)

- Object cached by client is fresh: web server sends a response message with **304** status code & empty body section
- Object cached by client is stale: web server sends a response message with **200** status code & object transfer

Combined use of expiration and validation mechanisms and heuristics

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure